

Benchmarking Failures in Tool-Augmented Language Models

Eduardo Treviño* Hugo Contant* James Ngai
Graham Neubig Zora Zhiruo Wang
Carnegie Mellon University
{eatrevin,hcontant}@andrew.cmu.edu

Abstract

The integration of tools has extended the capabilities of language models (LMs) beyond vanilla text generation to versatile scenarios. However, tool-augmented language models (TaLMs) often assume ‘perfect’ information access and tool availability, which may not hold in the real world. To systematically study TaLMs’ imperfections, we introduce the FAIL-TALMS benchmark, featuring two major failures: under-specified user queries and non-available tools. FAIL-TALMS contains 1,749 examples using 906 tools across 21 categories, including single- and multi-tool usage. We evaluate top-performing proprietary and open-source models, and find all current models except for Claude struggle to recognize missing tools or information. Further, to study possible mitigation of the failures, we enable real-time human interaction, named the Ask-and-Help (AAH) method, to provide missing information or replace non-functional tools. While AAH can help models solve tasks more correctly when queries are under-specified, it brings minimal benefit when complex tools are broken.¹

1 Introduction

Tools can greatly enhance language models (LMs) by facilitating their problem-solving process (Qin et al., 2023a; Mialon et al., 2023) and extending their abilities (Wang et al., 2024a). Given a user query, a tool-augmented language model (TaLM) can selectively call tools to gather more information and perform computation activities to accomplish the user’s request. Such TaLMs have been applied in various scenarios, including interacting with versatile knowledge bases (Lazaridou et al., 2022), real-world data (Xu et al., 2023), and even multi-modal information (Gupta and Kembhavi, 2022; Wang et al., 2024b). On the other hand, tools can deprecate over time (Qin et al., 2023b),

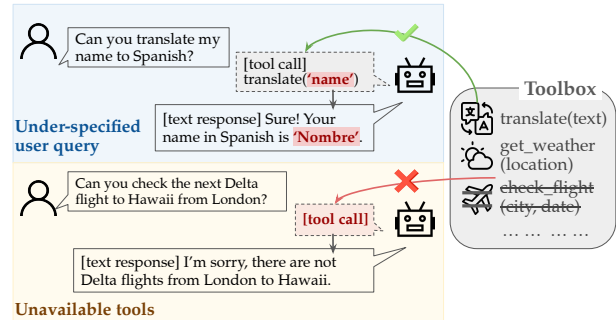


Figure 1: Illustration of two major TaLM issues. Left: The user provides *under-specified queries*, which may cause models to hallucinate or make false assumptions about the missing information, e.g., translate the “name” string instead of the user’s actual name. Right: Necessary *tools are unavailable*, e.g., missing `check_flight` causes TaLMs to lack the ability to solve the task.

suddenly break (Guo et al., 2024), or even return unpredictably false outputs (Sun et al., 2024).

Nonetheless, many approaches assume two idealized conditions for TaLM systems: (i) user queries are always sufficiently detailed for models to solve the task, and (ii) all necessary tools are available. In practice, however, these assumptions often do not hold, leading to failures like those depicted in Figure 1. In the first example, the user query is under-specified, and the model, while successfully calling the translation tool, lacks the necessary input, i.e., the user’s actual name. As a result, the model incorrectly translates the phrase “my name”, rather than the intended name. In the second case, the necessary `check_flight` tool to solve the task is unavailable, as denoted by the red cross. In these cases, existing TaLMs often hallucinate contexts or terminate without signaling failure or seeking alternative strategies, leading to sub-optimal behaviors.

To systematically study these practical failures and enable more robust TaLM systems, we introduce FAIL-TALMS — a benchmark designed to examine TaLMs under information insufficiency and tool unavailability (§3). We gather 906 real-

*Co-First Author; see Contribution Statement 7 for details.

¹<https://github.com/EduardoTrevino/fail-talms>

world tools across 21 categories directly from their host sources, and construct execution environments for all tools along with verifying test cases. Unlike existing benchmarks with limited tool calls via third-party platforms, our tool environment allows real-time and reproducible testing. With this collection of tools, we created 575 examples with perfect information and tool availability. We then transformed them into 599 and 575 examples with under-specified queries and unavailable tools, by removing key information from the queries and masking out necessary tools from the provided list, to study the two major failure modes mentioned above. Overall, FAIL-TALMS contains 1,749 queries across 906 tools from 21 categories.

We propose three evaluation metrics to study model performance: *pass rate* to measure task success, *awareness* of missing tools or information, and further, *unexpected outcomes* to capture when TaLM correctly yet unexpectedly solves a task.

We experiment with a series of top-performing LMs (§4): open-weight LLAMA 3 models from 8B, 70B to 405B, and proprietary models including CLAUDE and GPT. Our experiments reveal that most models struggle to identify the lack of tools or information needed to solve a task, except for CLAUDE with a 56% awareness rate, 28–54% higher than other models. Nonetheless, high awareness does not translate to higher pass rates. For example, GPT-4o achieves 4% higher pass rate than Claude-3.5-sonnet, despite scoring 44% lower in awareness.

Finally, to examine whether simple mitigation measures could address these issues, we study if a method enabling TaLMs to interact with humans, dubbed “Ask-and-Help” (AAH), could help obtain missing information or fulfill the function of unavailable tools (§5). We measure *interaction ratio* to see how often TaLM interacts with humans. AAH substantially improves pass rate particularly when user queries are under-specified, where the models actively interact with humans 21–61% of the time to gather missing information. However, this human assistance does not bring improvements when tools are unavailable, regardless of the tool functions are replaceable by humans or not, indicating room for better methodologies.

2 Problem Statement

A tool-augmented language model (TaLM) consists of (1) a backbone language model $\mathcal{M}$ and

(2) a set of tools $T = \{t_1, \dots, t_n\}$. Each tool t_i is a callable function (e.g., `calculator(expr)`, `document_retriever(query, docs)`). Given a natural language (NL) query q , the TaLM selects a series of tools $T^q \subseteq T$ to solve the query. For each chosen tool $t_i \in T^q$, the TaLM produces a tool-calling program $p_i^q = \mathcal{M}(q, T)$ that is executed then yielding output e_i^q . Finally, the TaLM produces the final answer r^q to query q based on all tool outputs $\{e_i^q\}$.

However, this classic TaLM pipeline may not successfully execute in practice due to two primary issues:

Under-specified Queries When the user query q is under-specified, either the subset of relevant tools T^q cannot be successfully identified, i.e., $\mathcal{M}_{\text{det}}(q, T) \not\equiv T^q$, or the set of tool-calling programs $P^q = \{p_i^q\}$ cannot be properly constructed due to insufficient information to determine the input arguments of tools.

Unavailable Tools Even when there is sufficient information, the tool may be unavailable due to reasons such as deprecated functions or server execution errors (e.g., server timeout or connection failure). In such cases, the tool execution fails or returns incorrect results, i.e., $\text{exec}(p) \not\rightarrow e$, leading to invalid or inaccurate tool execution outputs.

3 The FAIL-TALMS Benchmark

In this section, we first introduce the tool collection (§3.1) and benchmark curation processes (§3.2), present the data overview (§3.3), then establish the set of evaluation metrics (§3.4).

3.1 Tool Collection and Validation

Tool Collection We use Mixed Analytics² and collect 1,106 authorization-free tools, each with an URL, documentation of functionality and argument descriptions, and exemplar use cases.

Tool Validation To verify the successful executions of collected tools, we transform each tool instance into a callable Python function and synthesize multiple unit tests for it. More specifically, we provide the above-gathered information of each tool to GPT-4o and prompt it (see exact prompt in §A.1) to generate (i) Python file sending requests to the tool URL, and (ii) a JSON schema with the tool’s metadata, i.e., tool representation, and (iii)

²<https://mixedanalytics.com/blog/list-actually-free-open-no-auth-needed-apis/>

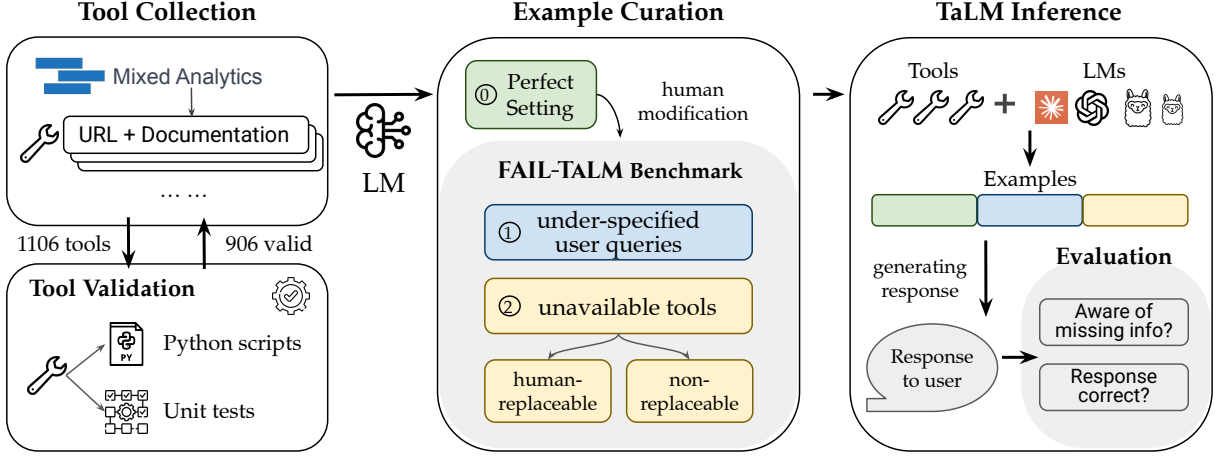


Figure 2: Visualization of the benchmark and tool environment construction (top), as well as the inference pipeline with awareness querying and human interaction phases (bottom).

unit tests for each tool. See examples of (i)–(iii) in §A.2, §A.3, and §A.4.

After tool environment construction, we validate if tools can (i) successfully execute and produce valid output and (ii) correctly pass all synthesized unit tests. We only keep tools that encounter no issues in (i) and (ii). Further, we maintain test cases to check tool availability in real-time, with an average response time of 1.47 seconds. We filter out tests having response times over 20 seconds to enable fast tool responses and test efficiency during inference. After this process, we collected a total of 906 valid tools.

3.2 Benchmark Creation

Given the tools, we now create queries that ask to solve certain tasks by using one or multiple tools.

3.2.1 The Standard ‘Perfect’ Setting

The standard ‘perfect’ setting adopted by most tool benchmarks assumes fully specified queries and the availability of all necessary tools to solve a given task. We refer to this ideal baseline scenario as the *perfect* setting, which serves as the foundation for generating the remaining data settings.

To generate a ‘perfect’ example with query and involved tools, we first construct a set of tool combinations to instantiate NL queries from, by pairing every tool with all other tools within the same category. This systematic approach ensures the usage of every possible tool combinations, instead of biasing over any specific tool.

To create an NL query for each given tool combination, we provide the tool information as collected in §3.1, as well as a one-shot (query, tools) example

as in §A.5 to demonstrate a query and two tools necessary to solve the query. We instruct GPT-4o to generate queries in realistic usage with content related to the tools’ functions, and include the model prompt in §A.6.

After this step, we perform an additional human validation step. During this step, human reviewers manually examine the generated queries and tool combinations to ensure that the queries are coherent, the arguments provided are valid, and the tool usage is contextually appropriate. We generate queries for all possible pairs of unique tool combinations in a given category, which yields 575 (query, tools) examples. This serves as the foundation for creating the rest of the benchmark, as illustrated in the *perfect* setting in Figure 2 the Example Generation module.

3.2.2 Under-Specified Queries

In real-world scenarios, queries are not always fully specified, and crucial details may be omitted. Hence, we create this data split to study whether the model can identify the missing information needed to construct tool calls. We refer to such queries, which maintain their semantic intent but lack essential details, as *under-specified queries*.

To create this setting, we modify the *perfect* queries by manually masking out key information required to define the input arguments for the relevant tools. For example, the standard query “What is the weather in Pittsburgh?” calls for the tool `Weather(location: str) → str`, which needs the location “Pittsburgh” as an argument. By removing “Pittsburgh,” the query becomes “What is the weather?”, which still implies

the use of the Weather tool but omits the specific location. We manually remove these critical arguments from the *perfect* queries, yielding a total of 599 under-specified queries. We generate more under-specified queries than perfect ones because a query may have more than one argument and can therefore be masked in multiple ways.

3.2.3 Unavailable Tools

In practice, tools required by TaLMs may not always be reliable, for example, tools can be susceptible to depreciation or errors (e.g., 404) especially when provided by third-party platforms with access limitations (Guo et al., 2024). This data split investigates how models perform when tools turn unexpectedly unavailable. Similarly, we manually decide which tools to remove and verify the quality of the modifications. Moreover, we study the distinction among tools, particularly in whether they can be easily replaced by an average human. We categorize data into two scenarios — *human-replaceable tools* and *non-replaceable tools*.

Human-replaceable tools features tools whose functions can be easily replaced by a normal human with minimal effort, such as calculating the value of $1 + 1$ with a calculator tool, or saying a random joke with the joke tool. Human-replaceable tools often have relatively easy functions.

Non-replaceable tools usually possess more complex functionalities that humans cannot easily replicate, such as complex calculations (e.g., `calculator(45465 * 5487)`) or simulating a rocket launch using the `rocket_simulator` tool.

For both unavailable tool scenarios, we manually classify a tool into human-replaceable or non-replaceable by asking ‘Can human replace the tool with minimal effort?’ If yes, the tool is placed into the *human-replaceable* setting; otherwise, it is placed in the *non-replaceable* setting. We yield 261 and 314 examples with unavailable tools that are human-replaceable and non-replaceable, respectively. In experiments, we provide all tools except the selected unavailable ones to the TaLM.

No Tools TaLMs may know certain unprovided tools, which we are unaware of due to their proprietary training data (Zhuang et al., 2023; Huang et al., 2024), we introduce a *no-tool* setting, where only NL query is provided. This setting serves as a baseline for measuring the model’s inherent knowledge. Under this setting we only provide the *perfect* setting queries to TaLM without any tool in-

formation. We run the same number of 575 queries as in the *perfect* set.

3.3 Benchmark Analysis

As shown in Figure 4, our FAIL-TaLMs benchmark spans 21 categories with 906 tools, most prominently featuring game, finance, and science, among other domains. Among the total of 1,749 examples, we have 575 *perfect* examples, 599 with *under-specified* queries, and 575 examples with unavailable tools — 261 of them are *human-replaceable*, and 314 are *non-replaceable*. See §B for detailed distribution of human- and non-replaceable tools in individual categories.

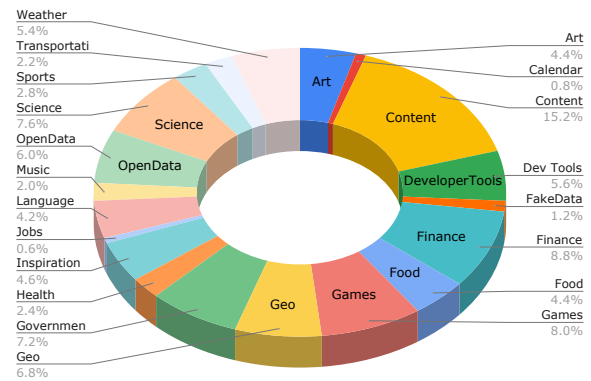


Figure 3: Category statistics of FAIL-TaLMs queries.

3.4 Evaluation Metrics

We introduce the evaluation metrics regarding task success, awareness of missing components, among other dimensions.

3.4.1 Correctness: Pass Rate

We adopt the pass rate metric from Qin et al. (2023b), which calculates the proportion of successful tasks. Specifically, we pass in the initial user query, the tool outputs, and the final TaLM response to a GPT-4o model, and ask it to evaluate if the final response solves the user query. GPT-4o grades responses binarily as ‘Pass’ (1) or ‘Fail’ (0). The final score is calculated by a majority vote across 5 GPT-4o graders. Refer to §C.1 for the detailed prompts and §D for human validation of GPT evaluation quality.

3.4.2 Awareness of Missing Components

Besides task completion, we also assess if TaLMs can identify the missing information or tools.

Information Awareness For *under-specified queries*, we evaluate whether the TaLM can detect insufficient information. We prompt the TaLM

to determine if the query provides enough information to complete the task by responding yes, idk (I don’t know), or no. A successful identification of insufficient information occurs when the model answers either idk or no. Formally, we define:

$$\text{Info Awareness} = \frac{\text{Number of idk's and no's}}{\text{Total number of examples}}$$

Tool Awareness When required tools are unavailable or non-functioning,³ we measure the TaLM’s ability to recognize this limitation. To evaluate this, we prompt the model with: “Do you have the right tools to complete the task?” and ask for a yes, idk, or no response. Similar to Information Awareness, successful identification of tool unavailability occurs when the model answers either idk or no. Unlike Information Awareness, however, this metric specifically targets the model’s recognition of tool-based limitations rather than informational gaps. For the non-replaceable setting, we thus have:

$$\text{Tool Awareness} = \frac{\text{Number of idk's and no's}}{\text{Total number of examples}}$$

See §C.2 for detailed prompts and evaluations.

3.4.3 Unexpected Success

In addition to cases where LMs correctly identify missing components and fail gracefully, an interesting scenario is when TaLMs unexpectedly solve the task correctly, despite lacking certain required information or tools.

Across examples with under-specified queries and unavailable tools, TaLMs are expected to respond idk or no to awareness questions. However, if the TaLM responds yes (to either information or tool awareness) and achieves a *pass rate* of 1, this outcome is noteworthy. We thus compute the unexpected success by:

$$\text{Unexpected Success} = \frac{\text{yes and pass rate} = 1}{\text{Total number of examples}}$$

3.4.4 Skipped Queries

A query is skipped if the model responds no to an awareness question. We explicitly prompt the TaLM to respond no if it confidently believes that it lacks the necessary tools or information to solve a given task, and inform it that this decision will skip the task. We calculate the ratio of skipped examples among all examples. A lower score indicates fewer skipped queries, and higher model confidence.

³Non-functioning tools are those that cannot fulfill the requested operation, either because they are not accessible or lack the functionality needed for the task at hand.

4 Experiments and Results

In this section, we introduce the experimental setup (§4.1) and TaLM performance without AAH (§4.2) and with (§5) human assistance via AAH.

4.1 Experimental Setup

We evaluated five models including the openweight Qwen-72B-Instruct and Llama models of various sizes (8B, 70B, and 405B); we also benchmark multiple closed API models: GPT-4o and Claude-3.5-Sonnet. We use the default temperature of $t = 1.0$ and sample $n = 1$ responses. We evaluate models on each split in FAIL-TALMS with the metrics specified in §3.4.

We report all results on FAIL-TALMS in Table 1.

4.2 Standalone TaLMs

Standard ‘Perfect’ Baseline In the standard ‘perfect’ setting, all models exhibited high awareness from 94–100%, correctly identifying that they had all the necessary information and tools to solve the tasks. Despite this high self-awareness, pass rates are lower than perfect — GPT-4o at 68%, Claude at 67%, Qwen at 54%, Llama 405 at 53%, Llama 70B at 31%, and Llama 8B at 28%; which correlate well with their general abilities in problem-solving.

Awareness of Missing Components Across settings with either under-specified queries or unavailable tools, Claude achieves substantially higher awareness of missing information — an average of 42%, which is 23–40% higher than other models, in particular, even 24% higher than the top-performing GPT-4o. Similarly, in both non-replaceable and replaceable tool settings, Claude significantly outperforms other models and achieves a 70% awareness, which is 34–68% higher than other models and 64% higher than GPT-4o.

Among Llama models, the medium-sized 70B model exhibits the highest awareness in the under-specified query split of 19%, which is 17% and 8% higher than its 8B and even 405B counterparts. This trend continues in the unavailable tool setting, where the 70B model attains 36% awareness, and gets 34% higher than both the 8B and 405B models. In comparison, the other open-source candidate, Qwen, obtains relatively low awareness around 3% with unavailable tools, mirroring the difficulty in recognizing missing components in weaker Llama models.

Setting	Claude-3.5-sonnet				GPT4o				Qwen-72B-Instruct			
	<i>PR</i>	<i>Aware</i>	<i>Unexp</i>	<i>Skip</i>	<i>PR</i>	<i>Aware</i>	<i>Unexp</i>	<i>Skip</i>	<i>PR</i>	<i>Aware</i>	<i>Unexp</i>	<i>Skip</i>
Perfect	0.67	0.94	0.00	0.00	0.68	1.00	0.00	0.00	0.54	0.94	0.00	0.00
Under-specified	0.31	0.42	0.24	0.08	0.36	0.18	0.33	0.05	0.40	0.08	0.31	0.05
Unavailable tools	0.25	0.70	0.03	-	0.28	0.06	0.05	-	0.15	0.03	0.04	-
non-replaceable	0.09	0.85	0.04	0.09	0.11	0.09	0.09	0.06	0.06	0.05	0.05	0.03
replaceable	0.41	0.54	0.01	-	0.44	0.03	0.01	-	0.23	0.00	0.02	-
No-tools	0.10	-	0.00	0.00	0.29	-	0.01	0.00	0.09	-	0.01	0.00

Setting	Llama 8B				Llama 70B				Llama 405B			
	<i>PR</i>	<i>Aware</i>	<i>Unexp</i>	<i>Skip</i>	<i>PR</i>	<i>Aware</i>	<i>Unexp</i>	<i>Skip</i>	<i>PR</i>	<i>Aware</i>	<i>Unexp</i>	<i>Skip</i>
Perfect	0.28	1.00	0.00	0.01	0.31	0.99	0.01	0.01	0.53	1.00	0.00	0.00
Under-specified	0.14	0.02	0.13	0.00	0.29	0.19	0.29	0.11	0.25	0.11	0.25	0.06
Unavailable tools	0.11	0.02	0.02	-	0.07	0.36	0.10	-	0.24	0.02	0.07	-
non-replaceable	0.03	0.02	0.03	0.00	0.04	0.55	1.00	0.38	0.12	0.02	0.12	0.02
replaceable	0.19	0.01	0.01	-	0.10	0.17	0.19	-	0.36	0.02	0.01	-
No-tools	0.00	-	0.00	0.00	0.37	-	0.02	0.00	0.28	-	0.06	0.00

Table 1: Performance of Claude, GPT, Qwen, and Llama models (8B, 70B, 405B) on FAIL-TALMs under uncertain and partial information settings. *PR* stands for pass rate, *Aware* refers to information/tool awareness, *Unexp* stands for unexpected outcomes, and *Skip* refers to skipped queries.

Is Awareness Related to Pass Rate? In the under-specified setting, all TaLMs except Claude, predominantly responded yes during the awareness assessment 82%–98% of the time, as shown in 1. This suggests the Llama, Qwen, and GPT-4o models often confidently proceed despite uncertainty being present. Therefore there is no clear correlation between awareness and pass rate, and more of a characteristic of individual models.

Similarly in the *unavailable tools* setting shown in Table 1, Claude identifies 64% more of the missing tools than GPT-4o, yet scores 3% lower in pass rate than GPT-4o, suggesting a better judgment in knowledge sufficiency yet less success in solving the task under sub-optimal settings.

Comparing Llama models with varied sizes, increasing model size from 8B to 70B increases the awareness score by 34%, yet further increasing model size to 405B drags down the awareness score by 8%, indicating decreased abilities to identify insufficient information, and behavior proceeding with the task even with insufficient content. Despite this loss in awareness, the pass rate increases up to 24% with the largest 405B model.

Overall, strong models such as GPT-4o and Llama-405B may overestimate their capabilities, engaging in tasks they cannot solve, while Claude and smaller Llama models often approach unsolvable tasks more cautiously.

Comparing Human Replaceable and Non-Replaceable Tools Compared to non-replaceable tools, all models provided with human-replaceable

tools show decreases in tool awareness by 1–34%. In the meantime, models present substantial increases in pass rate by 6–36%. Despite insufficient information, the nature of human-replaceable tools deceives the model into thinking they have sufficient information and thus can more easily solve the task. We conjecture this is due to the easier functionalities human-replaceable tools have, e.g., `get_weather(location)`, compared to complex tools such as `rocket_simulator()` which humans cannot easily replace.

Unexpected Success When Queries are Under-Specified Unexpected success should be minimal in all settings. This hypothesis holds in examples with unavailable tools, where all models achieve less than 5% of the cases correct. However, when user queries are under-specified, we surprisingly observe that all models obtain a substantial amount of unexpected successes, most prominently with GPT-4o solving 33% of the cases correctly.

We investigate this intriguing phenomenon and found models often refer to normal pragmatics and accidentally correctly assume the missing information to solve the task. For instance, given a user query “How is the stock performance today?” with the company name under-specified. However, the model may pragmatically assume the missing reference to be S&P 500 Index, since it is the most widely followed U.S. market benchmark. Despite the ambiguity, the model pragmatically fills in the missing information and solves the task correctly yet unexpectedly.

5 Human-Assisted TaLMs

5.1 The Ask-and-Help (AAH) Setting

One approach to alleviate TaLM failures under issues presented in FAIL-TALMS is to obtain assistance from humans. Therefore, to examine how much human intervention can alleviate TaLM failures, we also experiment in a setting where TaLMs can request human assistance as an interactive tool. We refer to this setup as Ask-and-Help (AAH).

The TaLM can choose to invoke AAH at any time during inference, by querying humans as calling a tool with a textual argument a , e.g., “What is your name” for the example in Figure 1 (top). The human then responds to a with additional information or solutions, e.g., “Mike”, which would be returned to the TaLM, much like the response from a traditional tool call. The TaLM then generates its final response based on human-provided information and other tool-calling or intermediate reasoning outputs. Since the pass rate is calculated using the final response of the TaLM, and AAH occurs within its normal tool calling process, our interpretation of our other metrics does not change from the basic setup.

Additional Evaluation: Interaction Ratio We also measure the number of examples where TaLM chooses to interact with humans via AAH, showing its awareness and willingness to seek help.

5.2 TaLM Performance with AAH

We explored the impact of offering TaLMs the ability to interact with a human via the AAH method on FAIL-TALMS. Effective use of AAH depends on the model’s ability to recognize when assistance is needed and to interact appropriately.

Interaction Improves Under-Specified Tasks

After being augmented with AAH, the *human-replaceable* setting is now equipped with sufficient information. Correspondingly, strong models, including GPT-4o, Claude, and Llama 405B show slight increases in PR with the human interaction by 1–2% as in Table 1 and Table 2. However, smaller models do not observably improve with human assistance, even with the missing tools that can be easily replaced by humans.

In contrast, the performance of AAH-assisted TaLM on the *under-specified query* setting shows significant pass rate improvements, by 25%, 30%, and 28% on GPT-4o, Claude, and Llama-405B. For Llama models with varying sizes, we also observe

larger improvements as the model size grows, increasing from 10% at 8B to 28% at 405B. Notably, the pass rate of Llama-405B with under-specified queries matched its perfect setting— a 53% pass rate. Moreover, Llama-70B surpasses its perfect setting pass rate by 2%. Similarly, Qwen also shows moderate gains 6–8% following the trend observed in the smaller Llama models.

For awareness, all models except Llama 405B increase by 3–9%, suggesting that enabling human interaction may affect models’ self-uncertainty assessment, leading to increased awareness of missing information. Lastly, the unexpected success increases by 7–28% across all TaLMs except Llama 70B, because without human assistance, success in this setting is otherwise unexpected.

Human Interaction versus Awareness All models generally interact with humans via AAH. Particularly when given under-specified queries, GPT and Claude models show substantially higher interaction rates than other models or than on other data splits, suggesting that they are eager to utilize human assistance to resolve incomplete information, as seen in Table 2.

Across all models and settings, we do not find clear associations between the awareness of missing components, versus the interaction rate to AAH. Rather, interactivity and awareness are more dependent on the model itself. In the *non-replaceable setting*, Claude’s awareness remained high at 85%, but its interaction rate was only 27%, indicating limited use of human assistance despite recognizing high uncertainty.

Despite recognizing high uncertainty (i.e., an 85% awareness), Claude does not proportionally increase its use of AAH nor achieve a higher pass rate via interaction, suggesting misalignment between self-uncertainty and the decision to seek help.

6 Related Works

TaLM Benchmarks Most benchmarks about tool-augmented LMs collect tools from existing platforms that allow offline (Yang et al., 2023; Xu et al., 2023) or online executions (Li et al., 2023; Chen et al., 2024), yet may bring degradation issues, where many tools become outdated over time (Guo et al., 2024). More recent datasets emphasize tool diversity and realistic use cases (Qin et al., 2023b; Patil et al., 2023; Li et al., 2023; Tang et al., 2023), yet still assume perfect information and tool availability. In contrast, our work studies failures

Setting	Claude-3.5-sonnet				GPT4o				Qwen-72B-Instruct			
	<i>PR</i>	<i>Aware</i>	<i>Unexp</i>	<i>Inter</i>	<i>PR</i>	<i>Aware</i>	<i>Unexp</i>	<i>Inter</i>	<i>PR</i>	<i>Aware</i>	<i>Unexp</i>	<i>Inter</i>
Perfect	0.67	0.94	0.00	-	0.68	1.00	0.00	-	0.54	0.98	0.00	-
Under-specified	0.61	0.51	0.31	0.61	0.61	0.21	0.47	0.58	0.48	0.15	0.34	0.29
Unavailable tools non-replaceable replaceable	0.25	0.68	0.03	0.23	0.28	0.04	0.06	0.12	0.15	0.03	0.05	0.14
	0.07	0.85	0.03	0.27	0.10	0.05	0.10	0.09	0.05	0.03	0.05	0.15
	0.43	0.51	0.03	0.19	0.45	0.02	0.02	0.15	0.29	0.00	0.05	0.13
No-tools	0.10	-	0.00	-	0.029	-	0.01	-	0.09	-	0.01	-

Setting	Llama 8B				Llama 70B				Llama 405B			
	<i>PR</i>	<i>Aware</i>	<i>Unexp</i>	<i>Inter</i>	<i>PR</i>	<i>Aware</i>	<i>Unexp</i>	<i>Inter</i>	<i>PR</i>	<i>Aware</i>	<i>Unexp</i>	<i>Inter</i>
Perfect	0.28	1.00	0.00	-	0.31	0.99	0.01	-	0.53	1.00	0.00	-
Under-specified	0.24	0.01	0.24	0.24	0.33	0.25	0.24	0.21	0.53	0.02	0.53	0.25
Unavailable tools non-replaceable replaceable	0.08	0.02	0.02	0.23	0.05	0.13	0.05	0.21	0.20	0.05	0.09	0.20
	0.03	0.03	0.03	0.26	0.05	0.21	0.05	0.18	0.16	0.09	0.15	0.16
	0.13	0.01	0.00	0.19	0.04	0.04	0.05	0.24	0.38	0.01	0.02	0.24
No-tools	0.00	-	0.00	0.00	0.37	-	0.00	-	0.28	-	0.06	-

Table 2: Performance of Claude, GPT, Qwen, and Llama models (8B, 70B, 405B) on FAIL-TALMS under uncertain and partial information settings with **AAH** assistance. *Inter* refers to interaction rate.

with under-specified queries and unavailable tools, and alleviates them via human interaction.

Tool Failures in Practice Many TaLM works assume a perfect tool execution environment and user query specification, which no longer holds when used in practice. On the one hand, NL queries are often under-specified (Min et al., 2020), necessitating models to ask more information to proceed with the task. On the other hand, tools can deprecate over time (Qin et al., 2023b), suddenly broken (Guo et al., 2024), or even return unpredictably false outputs (Sun et al., 2024). Some works propose to use LM as a neural simulator of tool execution (Kim et al., 2023; Guo et al., 2024) to maintain the perfect tool-availability assumption. Our work, instead, directly reveals and tackles both practical issues and proposes AAH as an attempted solution.

Human-Model Interactive Problem Solving

Many TaLM are designed to operate autonomously (Wang et al., 2024b) without seeking help from other sources such as human users. However, as the tasks become more complex and the environment runs more dynamically (Guo et al., 2024), it sometimes becomes theoretically impossible for the TaLM to finish the task itself. Human-in-the-loop comes as a useful technique (Mosqueira-Rey et al., 2023), especially in risk-critical tasks where human feedback or supervision is required. We thus employ this human interaction as an exploratory approach to alleviate our identified TaLM failures.

7 Conclusion

In this paper, we introduced FAIL-TALMS, a comprehensive benchmark designed to evaluate tool-augmented language models (TaLMs) under realistic conditions where user queries are under-specified or necessary tools are unavailable, consisting of 1,749 queries using 906 authorization-free tools across 21 categories. Our experiments with both proprietary models (GPT, Claude) and open-weights models (LLaMa series) revealed that most TaLMs struggle to recognize missing information or unavailable tools. To address this, we examine the Ask-and-Help (AAH) method, allowing TaLMs to interact with humans in real time to obtain missing information or substitute non-functional tools. While we find AAH improves the pass rate on under-specified queries, it has minimal impact when complex tools are unavailable.

Limitations

While FAIL-TALMS provides a substantial foundation for evaluating the practical failures of TaLMs, we primarily focus on two failure modes: under-specified queries and unavailable tools. Other potential risk issues, such as adversarial inputs, are not addressed and could be explored in future work. The AAH method involves human interaction, which may not be scalable or practical in all deployment scenarios due to concerns about latency, cost, or privacy. Implementing such a system in real-world applications would require careful

consideration of these factors.

Author Contributions

Eduardo Treviño * developed the AAH method, led all codebase development, constructed FAIL-TALMS benchmark, conducted experiments, ran analyses in results section, created evaluations metrics (i.e *Awareness*). Hugo Contant * API testing and experimentation lead to *Tool Validation* framework, assisted in analysis, experiments, and articulated *Non-replaceable vs Human-replaceable* setting. James Ngai assisted with code development, ran experiments, performed manual evaluations, and reviewed the manuscript. Zora Zhiruo Wang and Graham Neubig provided experimental guidance and revised the manuscript.

* Indicates Co-First Authors

References

- Zehui Chen, Weihua Du, Wenwei Zhang, Kuikun Liu, Jiangning Liu, Miao Zheng, Jingming Zhuo, Songyang Zhang, Dahua Lin, Kai Chen, and Feng Zhao. 2024. [T-eval: Evaluating the tool utilization capability of large language models step by step](#). *Preprint*, arXiv:2312.14033.
- Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. 2024. [Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models](#). *Preprint*, arXiv:2403.07714.
- Tanmay Gupta and Aniruddha Kembhavi. 2022. [Visual programming: Compositional visual reasoning without training](#).
- Yue Huang, Jiawen Shi, Yuan Li, Chenrui Fan, Siyuan Wu, Qihui Zhang, Yixin Liu, Pan Zhou, Yao Wan, Neil Zhenqiang Gong, and Lichao Sun. 2024. [Meta-tool benchmark for large language models: Deciding whether to use tools and which to use](#). *Preprint*, arXiv:2310.03128.
- Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. 2023. [An llm compiler for parallel function calling](#). *arXiv preprint arXiv:2312.04511*.
- Angeliki Lazaridou, Elena Gribovskaya, Wojciech Stokowiec, and Nikolai Grigorev. 2022. [Internet-augmented language models through few-shot prompting for open-domain question answering](#).
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. [Api-bank: A comprehensive benchmark for tool-augmented llms](#). *Preprint*, arXiv:2304.08244.
- Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. 2023. [Augmented language models: a survey](#).
- Sewon Min, Julian Michael, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2020. [AmbigQA: Answering ambiguous open-domain questions](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics.
- Eduardo Mosqueira-Rey, Elena Hernández-Pereira, David Alonso-Ríos, José Bobes-Bascarán, and Ángel Fernández-Leal. 2023. Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review*, 56(4):3005–3054.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. [Gorilla: Large language model connected with massive apis](#). *Preprint*, arXiv:2305.15334.
- Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Yufei Huang, Chaojun Xiao, Chi Han, Yi Ren Fung, Yusheng Su, Huadong Wang, Cheng Qian, Runchu Tian, Kunlun Zhu, Shihao Liang, Xingyu Shen, Bokai Xu, Zhen Zhang, Yining Ye, Bowen Li, Ziwei Tang, Jing Yi, Yuzhang Zhu, Zhenning Dai, Lan Yan, Xin Cong, Yaxi Lu, Weilin Zhao, Yuxiang Huang, Junxi Yan, Xu Han, Xian Sun, Dahai Li, Jason Phang, Cheng Yang, Tongshuang Wu, Heng Ji, Zhiyuan Liu, and Maosong Sun. 2023a. [Tool learning with foundation models](#).
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023b. [Toolllm: Facilitating large language models to master 16000+ real-world apis](#).
- Jimin Sun, So Yeon Min, Yingshan Chang, and Yonatan Bisk. 2024. [Tools fail: Detecting silent errors in faulty tools](#). *Preprint*, arXiv:2406.19228.
- Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. 2023. [Toolalpaca: Generalized tool learning for language models with 3000 simulated cases](#). *Preprint*, arXiv:2306.05301.
- Zhiruo Wang, Zhoujun Cheng, Hao Zhu, Daniel Fried, and Graham Neubig. 2024a. What are tools anyway? a survey from the language model perspective. *arXiv preprint arXiv:2403.15452*.
- Zhiruo Wang, Daniel Fried, and Graham Neubig. 2024b. [Trove: Inducing verifiable and efficient toolboxes for solving programmatic tasks](#).

Qiantong Xu, Fenglu Hong, Bo Li, Changran Hu, Zhengyu Chen, and Jian Zhang. 2023. [On the tool manipulation capability of open-source large language models](#).

Rui Yang, Lin Song, Yanwei Li, Sijie Zhao, Yixiao Ge, Xiu Li, and Ying Shan. 2023. [Gpt4tools: Teaching large language model to use tools via self-instruction](#). *Preprint*, arXiv:2305.18752.

Yuchen Zhuang, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. 2023. [Toolqa: A dataset for llm question answering with external tools](#).

A Benchmark Construction

In this section, we highlight exact details of the components required during the construction of the benchmark.

A.1 Tool Environment Construction

In this section, we describe the prompt we provide GPT-4o for constructing the tool environment. The instruction generates a tool request Python code, tool unit test cases, and a tool representation which includes metadata in JSON format based on a tools documentation. It iterates over all tool documents files and, for each tool, it generates these three files. All tools utilized are covered under the following licenses: Creative Commons, MIT License, GNU General Public License (GPL) 2.0 or later, Open Data Commons Open Database License, Database Contents License, openFDA, Apache 2.0 License, Massachusetts Department of Transportation Developers License Agreement, GNU GPLv3, and ISC License.

System Prompt

You are a helpful assistant designed to generate Python code, test cases, and metadata JSON files based on a tool documentation. Your task is to create Python functions to interact with all relevant tool URL's that a human might need based on the tool's documentation.

Ensure that the function names are properly formatted and include necessary parameters. Additionally, generate corresponding test cases to verify the tool's functionality, and create a JSON file with metadata about the tool.

Prompt

The following is documentation for a tool called "{tool_name}". Your task is to create a Python file "tool.py" to make requests to all the relevant tools that a human needs the functionality for based on the tool's documentation provided. Note: the tools function names should be lowercase and never start with a number.

Please ensure there are defaults in place (especially IDs or resource tags, etc., that are specific to the tool's URL). Additionally, ensure you create Test Cases separately to verify the tool's URL's work "tool_test.py".

Finally, create a metadata JSON file that provides metadata about the tool and all of its available endpoints "{tool_name}.json".

Here is an example tool file for a tool named artchicago: {seed_api_example}

Here is an example corresponding JSON file. Note how the names of the tool's in the `tool\_list` match the function names in the Python code calling the tool URL: {seed_json_example}

```
Now, please do this for the tool named
"{tool_name}". To capture your output
generation, be sure to bold the titles,
i.e., ### api.py then ```python or ```json
before the code block:
"""{documentation_content}"""
```

A.2 Tool Request

The following is an example of a tool request python file.

```
import requests
from typing import Optional, List

BASE_URL = "https://api.irail.be"

def stations(format: str = "json", lang: str = "en", ):
    """
    Retrieve a list of all stations.

    :param format: The response format (json, xml, jsonp).
    :param lang: The language of any text or names in the response.
    """
    url = f"{BASE_URL}/stations/"
    params = {
        'format': format,
        'lang': lang,
    }
    response = requests.get(url, params=params)
    try:
        return response.json()
    except Exception as e:
        return {"error": str(e), "response": response.text}

def liveboard(station: str, id: Optional[str] = None, date: Optional[str] = None, time: Optional[str] = None, arrdep: str = "departure", lang: str = "en", format: str = "json", alerts: bool = False, ):
    """
    Retrieve a liveboard for a specified station.

    :param station: The name of the station to query.
    :param id: Optional station ID.
    :param date: Date for query, formatted as ddmmyy.
    :param time: Time for query, formatted as hhmm.
    :param arrdep: Whether to retrieve departures or arrivals.
    :param lang: The language of the response.
    :param format: The output format (json, xml, jsonp).
    :param alerts: Whether to include alerts.
    """
    url = f"{BASE_URL}/liveboard/"
    params = {
        'station': station,
        'id': id,
        'date': date,
        'time': time,
```

```

        'arrdep': arrdep,
        'lang': lang,
        'format': format,
        'alerts': alerts
    }
    response = requests.get(url, params=params)
    try:
        return response.json()
    except Exception as e:
        return {"error": str(e), "response":
            response.text}

def connections(from_station: str, to_station:
    str, date: str, time: str, timesel: str =
    "departure",
                format: str = "json", lang: str =
                "en", typeOfTransport: str =
                "automatic", alerts: bool =
                False,
                results: int = 6, ):
    """
    Get routes between two stations, including
    realtime data on delays.

    :param from_station: The departure station.
    :param to_station: The destination station.
    :param date: Date for the query, formatted
        as ddmmyy.
    :param time: Time for the query, formatted
        as hhmm.
    :param timesel: Whether results should show
        arrivals or departures.
    :param format: The response format.
    :param lang: The language of the response.
    :param typeOfTransport: Types of transport
        to include.
    :param alerts: Include alerts or not.
    :param results: Number of results to return.
    """
    url = f"{BASE_URL}/connections/"
    params = {
        'from': from_station,
        'to': to_station,
        'date': date,
        'time': time,
        'timesel': timesel,
        'format': format,
        'lang': lang,
        'typeOfTransport': typeOfTransport,
        'alerts': alerts,
        'results': results
    }
    response = requests.get(url, params=params)
    try:
        return response.json()
    except Exception as e:
        return {"error": str(e), "response":
            response.text}

def vehicle(id: str, date: Optional[str] =
    None, format: str = "json", lang: str =
    "en", alerts: bool = False,
            ):
    """
    Retrieve information about a vehicle
    including stops and delays.

    :param id: The ID of the vehicle.

```

```

    :param date: Date for the query, formatted
        as ddmmyy.
    :param format: The response format.
    :param lang: The language of the response.
    :param alerts: Include alerts or not.
    """
    url = f"{BASE_URL}/vehicle/"
    params = {
        'id': id,
        'date': date,
        'format': format,
        'lang': lang,
        'alerts': alerts
    }
    response = requests.get(url, params=params)
    try:
        return response.json()
    except Exception as e:
        return {"error": str(e), "response":
            response.text}

def composition(id: str, format: str = "json",
    data: str = "", lang: str = "en",
                ):
    """
    Retrieve the composition of a train, i.e.,
    carriages and locomotives.

    :param id: The ID of the train.
    :param format: The response format.
    :param data: To get all raw unfiltered data
        use 'all'.
    :param lang: The language of the response.
    """
    url = f"{BASE_URL}/composition/"
    params = {
        'id': id,
        'format': format,
        'data': data,
        'lang': lang
    }
    response = requests.get(url, params=params)
    try:
        return response.json()
    except Exception as e:
        return {"error": str(e), "response":
            response.text}

def disturbances(format: str = "json",
    lineBreakCharacter: str = "", lang: str =
    "en",
                ):
    """
    Retrieve information about current
    disturbances.

    :param format: The response format.
    :param lineBreakCharacter: Character for
        line breaks in text.
    :param lang: The language of the response.
    """
    url = f"{BASE_URL}/disturbances/"
    params = {
        'format': format,
        'lineBreakCharacter': lineBreakCharacter,
        'lang': lang
    }
    response = requests.get(url, params=params)

```

```

try:
    return response.json()
except Exception as e:
    return {"error": str(e), "response":
           response.text}

```

A.3 Tool Representation

```

{
    "tool_name": "irail",
    "tool_description": "Tool to access railway
                        time schedules in Belgium, including
                        stations, liveboards, connections,
                        vehicles, disturbances, and more.",
    "title": "iRail API",
    "pricing": "FREE",
    "score": {
        "avgServiceLevel": 95,
        "avgLatency": 150,
        "avgSuccessRate": 98,
        "popularityScore": 9.0,
        "__typename": "Score"
    },
    "home_url": "https://api.irail.be",
    "host": "api.irail.be",
    "api_list": [
        {
            "name": "stations",
            "url":
                "https://api.irail.be/stations/",
            "description": "Retrieve a list of
                          all stations.",
            "method": "GET",
            "required_parameters": [],
            "optional_parameters": [
                {"name": "format", "type":
                  "STRING", "description":
                  "Response format",
                  "default": "json"},
                {"name": "lang", "type":
                  "STRING", "description":
                  "Language of response",
                  "default": "en"}
            ],
            "statusCode": 200
        },
        {
            "name": "liveboard",
            "url":
                "https://api.irail.be/liveboard/",
            "description": "Retrieve liveboard
                          for a station including arrivals
                          and departures.",
            "method": "GET",
            "required_parameters": [
                {"name": "station", "type":
                  "STRING", "description":
                  "Station name"}
            ],
            "optional_parameters": [
                {"name": "id", "type": "STRING",
                  "description": "Station ID"},
                {"name": "date", "type":
                  "STRING", "description":
                  "Date for query"},
                {"name": "time", "type":
                  "STRING", "description":
                  "Time for query"},
                {"name": "arrdep", "type":
                  "STRING", "description":

```

```

"Arrivals or departures",
"default": "departure"},
{"name": "lang", "type":
 "STRING", "description":
 "Language of response",
"default": "en"},
{"name": "format", "type":
 "STRING", "description":
 "Response format",
"default": "json"},
{"name": "alerts", "type":
 "BOOLEAN", "description":
 "Include alerts", "default":
 "false"}
],
"statusCode": 200
},
{
    "name": "connections",
    "url": "https://api.irail.be/c",
    "description": "Get routes between
                  two stations.",
    "method": "GET",
    "required_parameters": [
        {"name": "from", "type":
          "STRING", "description":
          "Departure station"},
        {"name": "to", "type": "STRING",
          "description": "Destination
                        station"}
    ],
    "optional_parameters": [
        {"name": "date", "type":
          "STRING", "description":
          "Date for query"},
        {"name": "time", "type":
          "STRING", "description":
          "Time for query"},
        {"name": "timesel", "type":
          "STRING", "description":
          "Arrivals or departures",
          "default": "departure"},
        {"name": "lang", "type":
          "STRING", "description":
          "Language of response",
          "default": "en"},
        {"name": "format", "type":
          "STRING", "description":
          "Response format",
          "default": "json"},
        {"name": "typeOfTransport",
          "type": "STRING",
          "description": "Type of
                        transport", "default":
          "automatic"},
        {"name": "alerts", "type":
          "BOOLEAN", "description":
          "Include alerts", "default":
          "false"},
        {"name": "results", "type":
          "INTEGER", "description":
          "Number of results",
          "default": 6}
    ],
    "statusCode": 200
},
{
    "name": "vehicle",
    "url":
        "https://api.irail.be/vehicle/",

```

```

"description": "Retrieve information
    about a vehicle.",
"method": "GET",
"required_parameters": [
    {"name": "id", "type": "STRING",
     "description": "Vehicle ID"}
],
"optional_parameters": [
    {"name": "date", "type":
     "STRING", "description":
     "Date for query"},
    {"name": "lang", "type":
     "STRING", "description":
     "Language of response",
     "default": "en"},
    {"name": "format", "type":
     "STRING", "description":
     "Response format",
     "default": "json"},
    {"name": "alerts", "type":
     "BOOLEAN", "description":
     "Include alerts", "default":
     "false"}
],
"statusCode": 200
},
{
    "name": "composition",
    "url":
        "https://api.irail.be/composition/",
    "description": "Retrieve the
        composition of a train.",
    "method": "GET",
    "required_parameters": [
        {"name": "id", "type": "STRING",
         "description": "Train ID"}
    ],
    "optional_parameters": [
        {"name": "format", "type":
         "STRING", "description":
         "Response format",
         "default": "json"},
        {"name": "data", "type":
         "STRING", "description":
         "Raw or filtered data",
         "default": ""},
        {"name": "lang", "type":
         "STRING", "description":
         "Language of response",
         "default": "en"}
    ],
    "statusCode": 200
},
{
    "name": "disturbances",
    "url":
        "https://api.irail.be/disturbances/",
    "description": "Retrieve information
        about current disturbances.",
    "method": "GET",
    "required_parameters": [],
    "optional_parameters": [
        {"name": "format", "type":
         "STRING", "description":
         "Response format",
         "default": "json"},
        {"name": "lineBreakCharacter",
         "type": "STRING",
         "description": "Line break
         character", "default": ""},

```

```

        {"name": "lang", "type":
         "STRING", "description":
         "Language of response",
         "default": "en"}
    ],
    "statusCode": 200
}
]
}

```

A.4 Python Unit Test Example

```

import unittest
from api import stations, liveboard,
connections, vehicle, composition,
disturbances

class TestIRailAPI(unittest.TestCase):

    def test_stations(self):
        response = stations()
        self.assertIn('station', response)

    def test_liveboard(self):
        response = liveboard('Gent-Sint-Pieters')
        self.assertIn('station', response)

    def test_connections(self):
        response =
            connections('Gent-Sint-Pieters',
                       'Mechelen', '23082024', '1130')
        self.assertIn('connection', response)

    def test_vehicle(self):
        response = vehicle('BE.NMBS.IC3033')
        self.assertIn('vehicle', response)

    def test_composition(self):
        response = composition('S51507')
        self.assertIn('composition', response)

    def test_disturbances(self):
        response = disturbances()
        self.assertIn('disturbance', response)

if __name__ == '__main__':
    unittest.main()

```

A.5 Query-tool Example

Below is an example of a query-tool interaction involving tools from the Met Museum and the Art Institute of Chicago.

```

{
    "tool_list": [
        {
            "category_name": "Art",
            "tool_name": "metmuseum",
            "function_name": "search_objects",
            "tool_description": "Search for
                objects in the Met's collection",
            "required_parameters": [
                {
                    "name": "q",
                    "type": "STRING",
                    "description": "Search term",

```

```

        "default": "Impressionist
        paintings"
    },
    ],
    "optional_parameters": [
        {
            "name": "departmentId",
            "type": "INTEGER",
            "description": "ID of the
            department",
            "default": "11"
        }
    ],
    "method": "GET",
    "template_response": {
        "total": "int",
        "objectIDs": ["int"]
    }
},
{
    "category_name": "Art",
    "tool_name": "artchicago",
    "function_name": "artworks_search",
    "tool_description": "Search artworks
    in the Art Institute of Chicago
    data in the aggregator. Artworks
    in the groups of essentials are
    boosted so they'll show up
    higher in results.",
    "required_parameters": [
        {
            "name": "q",
            "type": "STRING",
            "description": "Your search
            query.",
            "default": "monet"
        }
    ],
    "optional_parameters": [
        {
            "name": "size",
            "type": "INTEGER",
            "description": "Number of
            results to return.
            Pagination via
            Elasticsearch
            conventions.",
            "default": "10"
        },
        {
            "name": "sort",
            "type": "STRING",
            "description": "Used in
            conjunction with query to
            sort results.",
            "default": ""
        }
    ],
    "method": "GET",
    "template_response": {
        "pagination": {
            "total": "int",
            "limit": "int",
            "offset": "int",
            "total_pages": "int",
            "current_page": "int"
        },
        "data": [
            {
                "id": "int",

```

```

        "title": "str",
        "artist_display": "str",
        "place_of_origin": "str",
        "date_display": "str",
        "medium_display": "str",
        "dimensions": "str",
        "thumbnail": {
            "alt_text": "str",
            "width": "int",
            "height": "int",
            "iiif_url": "str"
        }
    }
},
{
    "query": "I want to find Impressionist
    paintings in the European Paintings
    department in the Met's collection.
    Additionally, can you find artworks
    related to Monet in the Art Institute
    of Chicago?",
    "query_id": 2
}
]

```

A.6 Perfect Queries Prompt

Perfect queries are constructed by combining different components in the Tool Environment. The code processes the Tool Requests, which are Python files that handle interactions with tools, and Tool Representations, which contain important information about the tools, such as required parameters, optional parameters, and expected responses. The code systematically pairs tools from the same category and generates. We prompt gpt-4o-2024-08-06 to generate a query combining these tools.

Prompt

Below I have attached 2 Tools "{tool1}" and "{tool2}", which are Python files that make requests to the tools from the "{category_folder}" category, and their corresponding metadata JSON files that provide additional information about the tools, as well as unit tests that have been run on these tools. Utilize the parameters used in these unit tests, and the information about the tools to help you with your task. Your task is to create a tool-question JSON file that asks a question a human would ask. Note: For the tool-question JSON file, be sure to include the name of the tool function from the Python files inside the{tool_list}; they should be the same name and format as the function provided in the Python code.

```

"{tool_1}" tool: {tool_1_python_request}
"{tool_1}" unittest: {tool_1_python_unittest}
"{tool_1}" tool metadata:
    {tool_1_json_representation}

```

```

"{tool_2}" tool: {tool_2_python_request}

```

```
"{tool_2}" unittest:{tool_2_python_unittest}
"{tool_2}" tool metadata:
    {tool_2_json_representation}
tool-question JSON example: {query-tool_example}
```

B Statistical Details of FAIL-TALMS

We study the distinction of human-replaceable and non-replaceable tools in §3.3. Here we present the detailed distribution of these two types of tools across the 21 categories involved in our FAIL-TALMS.

C Model-based Evaluation

C.1 Pass Rate

The pass rate metric evaluates whether the assistant's response successfully fulfills the user's instruction. The evaluation is performed by a grader model that assesses the assistant's reply and determines a pass or fail outcome. To ensure reliability, the evaluation is conducted multiple times (up to 5 attempts), and a majority voting mechanism is used to decide the final result. If the majority of evaluations result in a pass, the assistant's response is considered a pass; otherwise, it's a fail.

Prompt:

System Message:

You are a grader. You will be given a user's instruction, assistant's answer, and the API's accessed along with their responses. Based on the answer given, determine if the instruction was completed or not. Always begin your response with either 'Pass' or 'Fail'.

Prompt:

Below I have attached a user's instruction, assistant's answer, and attached API's accessed with their responses. Did the assistant's answer complete the instruction given? Begin your response with either 'Pass' or 'Fail'.

Query: {query_text}

Answer: {assistant_reply}

API's accessed and responses:
{function_context}

C.2 Information Awareness Prompt

The system is tasked with determining whether it can gather, infer, or has enough information to answer the user's query based on the following factors:

1. Existing knowledge of the world, independent of the tools.

2. The functionality of the available tools that the system has access to.
3. A combination of both existing knowledge and tool functionality.

When evaluating the query, the system considers whether:

- The query provides enough information for a direct answer.
- The available tools can assist in obtaining or clarifying the necessary information.
- A combination of the tool's functionality and existing knowledge can help infer, gather, or produce the necessary information.

Based on this evaluation, the system responds as follows:

- **Yes:** The system has all the information needed or can gather or infer it using either existing knowledge, available tools, or both. In this case, the system will attempt to answer.
- **IDK:** The system is uncertain but willing to attempt an answer using its knowledge, tools, or a combination of both.
- **No:** The system cannot answer the query with the available knowledge, tools, or any combination of both, and it will skip answering.

In all responses, the system must begin with **Yes**, **IDK**, or **No**, followed by a brief explanation of the reasoning. If the response is **No**, the system skips the query. If the response is **Yes** or **IDK**, the system will attempt to provide an answer.

Prompt:

System message:

Your task is to determine if you can gather, infer, or have all the information needed to answer the user's query using:

1. Your existing knowledge (regardless of the tools).
2. The functionality of the available tools you have access to.
3. A combination of both.

When evaluating the query, consider whether:

The query provides enough information for you to answer directly.

The available tools can help you obtain the necessary information or clarify the query.

Or if a combination of a tool's functionality and your existing knowledge helps you infer, gather, or have the necessary information you need to answer.

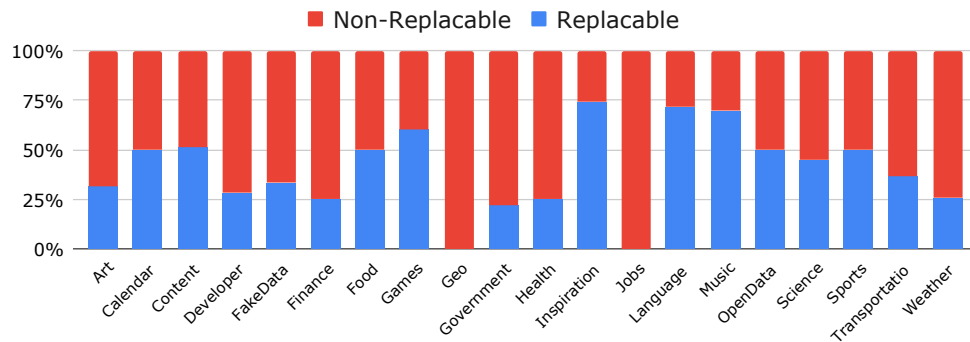


Figure 4: Distribution of human-replaceable and non-replaceable tools in each category in FAIL-TALMs.

Based on this evaluation:

Respond with 'Yes' if you can gather, infer, or have all the information needed to answer the query using your knowledge, the tools, or both.

Respond with 'IDK' if you are unsure but are willing to attempt an answer.

Respond with 'No' if you cannot answer the query at all, even with your knowledge and the tools available. This means the query is completely unanswerable given the current resources.

Important: Always start your response with 'Yes,' 'No,' or 'IDK,' followed by a brief explanation of your reasoning. If you respond with 'No,' you will skip attempting to answer the query. If you respond with 'Yes' or 'IDK,' you are willing to try to answer.

Prompt:

Based on the user's query, your knowledge of the world, and the functionality of the available tools, determine if you can gather, infer, or have all the information needed to answer the request. Remember: Start with 'Yes,' 'No,' or 'IDK,' followed by an explanation. 'Yes' means you have enough information, you can infer it, or can obtain it using the tools, and you will attempt to answer. 'IDK' means you are uncertain but willing to try using your knowledge, tools, or a combination of both. 'No' means you cannot answer the query with your knowledge, the available tools, or a combination of both and you will skip it. Query: {query}, Your knowledge of the world, Available Tools and Their Functionalities: {functions}

Tool Awareness Prompt The system is tasked with determining whether it has the necessary tools, knowledge, or a combination of both to answer the user's query. The evaluation is based on the following considerations:

1. The system's existing knowledge, independent of the available tools.
2. The functionality of the tools accessible to the

system.

3. A combination of both the existing knowledge and tool functionality.

When evaluating the query, the system considers whether:

- The functionality of the tools can help obtain the necessary answer.
- The system's existing knowledge allows for directly answering the query.
- A combination of tool functionality and existing knowledge is sufficient to provide an answer.

Based on this evaluation, the system will respond as follows:

- **Yes:** The system has sufficient knowledge, tools, or a combination of both, and will attempt to answer.
- **IDK:** The system is uncertain but willing to try answering using its knowledge, tools, or both.
- **No:** The system cannot answer the query, even with the available knowledge, tools, or a combination of both, and will skip attempting to answer.

In all cases, the system must begin its response with **Yes**, **IDK**, or **No**, followed by a brief explanation. If the response is **No**, the system will skip the query. For **Yes** or **IDK**, the system will proceed to answer the query.

Prompt:

System Message:

Your task is to determine whether you can answer the user's query using:

1. Your existing knowledge (regardless of the tools).
2. The functionality of the available tools you have access to.
3. A combination of both.

When evaluating the query, consider whether:
The functionality of the tools can help you obtain the answer.

Your existing knowledge allows you to answer directly.

Or if a combination of a tool's functionality and your existing knowledge helps you obtain the answer.

Based on this evaluation:

Respond with 'Yes' if you can answer the query using your knowledge, the tools, or a combination of both.

Respond with 'IDK' if you are unsure but are willing to attempt an answer.

Respond with 'No' if you cannot answer the query at all, even with your knowledge and the tools available, or a combination of both. This means the query is completely unanswerable given the current resources, and you will skip attempting to answer the query.

Important: Always start your response with 'Yes,' 'No,' or 'IDK,' followed by a brief explanation of your reasoning. If you respond with 'No,' you will skip attempting to answer the query. If you respond with 'Yes' or 'IDK,' you are willing to try to answer.

Prompt:

Based on the available tools' functionality and your knowledge of the world, determine whether you have the necessary tools, knowledge, or a combination of both to answer the query. Start with 'Yes,' 'No,' or 'IDK,' followed by an explanation. 'Yes' means you have the knowledge, tools, or a combination of both, and you will attempt to answer. 'IDK' means you are uncertain but willing to try to answer. 'No' means you cannot answer the query with your knowledge, the available tools, or a combination of both, and you will skip it. Query: {query}, Your knowledge of the world, Available Tools: {functions}

D Human Agreement with Model-Based Evaluation

In §3.4 we propose to use model-based evaluation to examine the correctness of model-generated responses, i.e., *pass rate*. To validate the reliability of GPT-4o's pass rate evaluation, we analyzed 95 examples randomly sampled from GPT-4o's responses on the perfect setting and compare the pass rate results between human evaluators and GPT-4o. The comparison showed an 83.2% agreement in the correctness of model-generated final responses

between GPT-4o and human evaluators. This high level of agreement indicates that GPT-4o provides evaluation results similar to those of humans, making it a credible evaluator capable of simulating human assessment of pass rates.

E Confidence Estimation of Results

While our benchmark contains a total of 1,749 examples, we conduct power analysis and result confidence estimations to further verify its effectiveness in supporting the conclusions drawn in our experiment sections.

First, we conducted significance testing and power analysis to determine the minimum number of examples required to detect meaningful differences between our experimented models with adequate statistical power. We found that at most 92 queries are sufficient to achieve reliable detection of meaningful differences between configurations with medium effect size ($w = 0.5$), a significance level of $\alpha = 0.005$, and a statistical power of 0.90. Our dataset contains 261—599 examples in each data split, which is sufficiently higher than 92, suggesting that it can guarantee reliable and significant results.

Moreover, to enhance the transparency and robustness of our evaluation results, we have included confidence intervals for our key metrics (pass rate, awareness, unexpected outcomes, and skipped queries) across all settings and models. Specifically, we calculate the 95% confidence intervals using the Wilson score interval for proportions and report them in [Table 3](#).

Metric	Model	Setting	Value (%)	N	Confidence Interval (%)
Pass Rate	GPT-4o	under-specified	36.0	599	[32.2, 39.9]
	Claude-3.5-sonnet	perfect	67.0	575	[63.1, 70.8]
	LLaMa-70B	unavailable tools (non-replaceable)	12.0	314	[8.6, 16.2]
Awareness	GPT-4o	under-specified	18.0	599	[15.0, 21.3]
	Claude-3.5-sonnet	under-specified	42.0	599	[38.1, 46.0]
	LLaMa-70B	unavailable tools	36.0	575	[32.1, 40.1]

Table 3: Confidence intervals of model performance.